Önálló laboratórium beszámoló

Távközlési és Mesterséges Intelligencia Tanszék

Név:	**Orbán Levente**
	orban.levente.laszlo@gmail.com
Neptun-kód:	**CZ6NDB**
Szakirány:	**Szoftverfejlesztés**
Konzulens:	**Németh Gábor**
	nemethgab@tmit.bme.hu
Konzulens:	

Téma címe: Interaktív SKI-kombinátor kalkulus fordító és vizualizáló

Feladat:
A hallgató feladata egy webes eszköz (Combinator Explorer) elkészítése, amely SKI-kifejezések esetén megmutatja a redukciós lépéseket, vizualizálja a redexeket, és felrajzolja a kiértékelési fát. A redukció során többféle kiértékelési stratégiát támogat, valamint az SKI-kifejezéseket lefordítja λ-kalkulusba. Az eszköz hidat képez a formális elmélet és a gyakorlati megvalósítás között: oktatási segédeszköz, amely mélyíti a funkcionális programozás, illetve a fordítóprogramok működésének megértését.

Tanév: 2025/26. tanév, II. félév

1. A laboratóriumi munka környezetének ismertetése, a munka előzményei és kiindulási állapota

1.1. Bevezető

Az elmúlt három évtizedben egyre nagyobb jelentőségre tesz szert a funkcionális programozás az iteratív és objektumorientált paradigmákkal szemben. Az figyelhető meg, hogy azok a programozási nyelvek, amelyek tradícionálisan objektumorientált szemléletűek voltak (például C++, Java), azokat napjainkra jelentős számú funkcionális elemmel bővítették (anonim függvények, lista komprehenzió, első osztályú függvények, etc.). Továbbá azok a nyelvek, amelyek az elmúlt évtizedben tettek szert népszerűségre (például Rust) már a kezdetektől fogva egy multiparadigmás nézettel lettek tervezve. Ezeket az új eszközöket viszont – mind a már piacon lévő mérnökök továbbképzése, mind a felsőoktatásban résztvevők oktatása szempontjából – pedagógiailag még a mai napig nehéz kielégítően oktatni. A funkcionális paradigma – illetve annak a matematikai alapjai – idegenek lehetnek a mérnöki beállítottságú közönség számára.

A jelen írásos beszámolóban bemutatott szoftver ezt a pedagógiai hiányt próbálja pótolni: egy olyan vizuális, interaktív, webes eszközt ad a pedagógusok kezébe, amellyel a funkcionális programozást egészen a matematikai alapokig visszamenően lehet hatékonyabban oktatni, hozzá pontosabb intuíciót építeni.

A választott megközelítés (vizuális, webes program) nem egyedi. Számtalan más olyan témakörhöz, amelyek tradícionálisan pedagógiai kihívásokat jelentettek (generált gépi kód, vagy reguláris kifejezések elemzése) készült hasonló alapötletekkel rendelkező szoftver [aref: compiler explorer, regex101].

A jelen beszámolóban bemutatott megközelítés a modern funkcionális nyelvek matematikai alapjait kívánja megcélozni (többek között redukciós láncok és gráfok, kiértékelési sorrendek, λ- és SKI-kalkulus). Ezen alaptudás ismeretében a legtöbb modern funkcionális nyelv (ML-család, Lisp-család) értelmezőjének a működése – legalábbis magas szinten – könnyebben elsajátítható.

1.2. Elméleti összefoglaló

A funkcionális programozási nyelvek alapjaikban a (típus nélküli) λ-kalkulusra építenek[1]. A λ-kalkulus egy általános célú, Turing-teljes [aref] programozási nyelv. Az egyik legegyszerűbb ilyen nyelv, hiszen csak három konstrukcióra épít (lambda-kifejezések, lambda-absztrakciók, és applikációk). A λ-kalkulus nyelvtanának formális definíciója az 1. ábrán látható.

$$
\begin{array}{lcl}
\langle \lambda\text{-}\textit{kifejezés} \rangle & ::= & \langle \textit{változó} \rangle \\
 & | & \langle \lambda\text{-}\textit{absztrakció} \rangle \\
 & | & \langle \textit{applikáció} \rangle \\
 & & \\
\langle \lambda\text{-}\textit{absztrakció} \rangle & ::= & \text{`}\lambda\text{'}\ \langle \textit{változó} \rangle\ \text{`.'}\ \langle \lambda\text{-}\textit{kifejezés} \rangle \\
 & & \\
\langle \textit{applikáció} \rangle & ::= & \langle \lambda\text{-}\textit{kifejezés} \rangle\ \langle \lambda\text{-}\textit{kifejezés} \rangle
\end{array}
$$

1. ábra. A λ-kalkulus szintaktikája

A λ-absztrakció segítségével tudunk függvényeket definiálni, amelyeknek paramétereket az applikáció konstrukciójával tudunk adni. Ha kicsit módosítjuk a nyelvtanunkat, hogy szám konstansokat, és egyéb kényelmes konstrukciókat megengedjen, akkor írhatjuk például:

$$
\begin{array}{l}
\lambda x.(5 * x) \\
(\lambda x.(5 * x))(3) \\
15
\end{array}
$$

Az első sorban definiálunk egy függvény, amely ötszörözi a bemenetét. A második sorban meghívjuk ezt a függvényt a 3 bemenettel. Az utolsó sorban az applikáció eredménye látható. Fontos megjegyezni, hogy az applikáció bal-asszociatív.

[1] Tekinthetőek a λ-kalkulus kibővített, változatának (legyen szó típusokról, vagy let-bindingokról, etc.). Példa erre az ML-család (Standard ML, Caml, Haskell), amelyek közelítőleg felfoghatóak úgy, mint a λ-kalkulus polimorf Hindley-Milner típusrendszerrel való kiegészítése.

A λ-kalkulus sok szempontból érdekes, viszont a gyakorlatban ritkán használják. Elterjedtebb az úgynevezett SKI-kalkulus, amely tekinthető a λ-kalkulus változómentes variánsának. Három úgynevezett *kombinátorból* áll, melyeket az alábbi λ-kalkulus kifejezésekkel definiálhatunk:

$$\begin{aligned} \mathsf{I} &= \lambda x.x \\ \mathsf{K} &= \lambda x.(\lambda y.x) \\ \mathsf{S} &= \lambda x.(\lambda y.(\lambda z.xz(yz))) \end{aligned}$$

Ezeket a kombinátorokat többszörösen egymásba ágyazva (applikáció révén) komplex függvényeket is ki tudunk fejezni. Pontosabban az egymásba ágyazások egy fa struktúrát definiálnak, amely az Abstract Syntax Tree (AST) analógja.

1.3. A munka állapota, készültségi foka a félév elején

Ebben a részben lehet megadni a korábbi félévekben elvégzett munkát is. Így világosan elkülöníthető az aktuális félévtől. Nem kell hosszasan írni, de ne legyen felsorolás sem. Egy bekezdésben kellene itt leírni, hogy foglalkoztál-e már ezzel a témával, a tanszéken dolgozott-e már valaki rajta. Írd le azt is, hogy mit kaptál kézhez hozzá segítségként.

A jelen félév előtti szünetben már elkezdtem feldolgozni a témához releváns irodalmat. A 2025/26/I. félévben egy olyan tárgyat hallgattam, amelyen jelentős szerepe volt a λ-kalkulusnak, kategória elméletnek. Ezek az előadások nagyban hozzájárultak az előzete tudás megszerzéséhez. A konzulensem megannyi releváns könyvet ajánlott feldolgozásra, mint például [aref: smullyan, wolfram].

2. Az elvégzett munka és az eredmények ismertetése

2.1. Az irodalom feldolgozása

A félév első heteiben a munka fókuszában a releváns szakirodalom, illetve a már létező fordítóprogramok és interpreterek működésének tanulmányozása állt. Kiemelt figyelmet fordítottam a λ-kalkulus és a SKI kombinátor logika elméleti hátterének felfrissítésére, valamint ezen elméleti modellek számítógépes reprezentációinak megismerésére.

A modern interpreterek esetében a hagyományos, fa alapú AST sokszor memóriatöredezettséghez és lassú futáshoz vezet. Ennek elkerülése érdekében a Flat-AST (laposított AST) reprezentációs stratégia mellett döntöttem, melyet a memóriakezelés szempontjából hatékony aréna-allokációs technikával egészítettem ki. A redukciós logikák implementálásához a Graph Reduction Machine (G-machine) koncepcióját vizsgáltam meg, amely a fa-alapú kiértékeléssel szemben a kifejezések megosztását (sharing) teszi lehetővé, elkerülve a redundáns számításokat. Emellett fel kellett térképeznem a Rust programozási nyelv – melyben az egész projektet implementáltam – által biztosított WebAssembly (WASM) fordítási lehetőségeket és a lexikális elemzők (lexerek) valamint szintaktikai elemzők (parserek) írásának modern megközelítéseit.

2.2. Az architektúra megtervezése

Az architektúra tervezésének folyamatában fordítóprogramokból merítettem inspirációt, különös tekintettel a moduláris felépítésre és az úgynevezett Separation of Concerns elvére.

A nyelvtanok egyszerűsége miatt a lexert és a parsert egybevontam. Két redukciós gépet implementáltam: egyet ami λ-kalkuluson, egyet ami SKI-kalkuluson operál. A két parser és a hozzájuk tartozó redukciós gépekre egy egységként tekintettem, ezek alkották a backend könyvtárat.

A frontend és a backend közötti tiszta kommunikáció megteremtése érdekében a Homlokzat (Facade) programtervezési mintát alkalmaztam. Ez az API réteg (`lexor-api`) elrejti a redukciós gépek komplexitását, és egy egységes interfészt biztosít a webes felület számára.

A frontend terén az úgynevezett *Elm-architektúrát* követtem, amely egy üzenetsor-alapú architektúra. Ez a megközelítés három részre bontja a frontendet:

- A kezdeti `modelre`, amely az applikáció állapotát írja le. Ez az állapot nem mutábilis, tehát ahogy a felhasználó interaktál vele, mindig egy teljesen új állapot jön létre.
- A `view` függvényre, amely a modell megjelenését írja le. A felhasználói interakciókat itt alakítom át üzenetekké.
- Az `update` függvényre, amely az üzenetsort kiolvasva végrehajtja a felhasználó összes parancsát a modellen, egy új modellt létrehozva ezáltal.

2.3. A redukciós gépek implementálása

A redukciós gépek implementálását a követelmények elemzésével és a belső reprezentáció kialakításával kezdtem. A G-machine megközelítésre építettem mind a λ-kalkulus, mind a SKI-kalkulus esetében. A teljesítmény optimalizálása érdekében egy egyedi memóriakezelőt (Arena allocator) alkalmaztam létre, amely Flat-AST struktúrában tárolja a kifejezéseket.

Mivel a szoftver célja az oktatás támogatása, több redukciós stratégiát is támogatnom kellett (például Normal order, Applicative order, lusta kiértékelés). A kialakításban különösen figyeltem arra, hogy a redukciós logika moduláris maradjon, így az egyes stratégiák könnyen cserélhetők és összehasonlíthatók anélkül, hogy a motor alapvető működését módosítani kellene.

A G-machine a kombinátorok redukcióját egy állapotgép segítségével végzi. A kiértékelés során a veremben (spine) összegyűlt argumentumokat a gép a megfelelő kombinátor szabályai szerint helyettesíti be (például az S, K, I kombinátorok esetében). A redukciós lépéseket végrehajtó Rust implementáció, különös tekintettel a memóriakezelőt használó gráf-mutációkra, a 3.2. szakaszban, a csatlakozó dokumentumok között tekinthető meg.

2.4. A tesztelő orákulum kialakítása

A szoftver helyességének verifikálására a hagyományos egységtesztek mellett a Property-based testing (tulajdonság-alapú tesztelés) módszertanát alkalmaztam. Ennek során a teszt-keretrendszer automatikusan generál valid, véletlenszerű λ- és SKI-kifejezéseket, majd megvizsgálja a rendszer invariáns tulajdonságait.

A redukciós gépek kimenetének validálásához egy tesztelő orákulumot alakítottam ki. Az orákulum egy már bizonyítottan helyesen működő, külső interpreter, amelyet egy elszigetelt Podman konténerben futtattam le a biztonság és a determinisztikus környezet garantálása érdekében. A tesztelési folyamat során a `lexor` és a Podman konténerben futó orákulum is kiértékelte ugyanazt a generált kifejezést, a teszt-keretrendszer pedig a két kimenet ekvivalenciáját vizsgálta Black-box (fekete doboz) tesztelés formájában.

2.5. A vizuális redukciós elemek létrehozása

Egy oktatási eszköz esetében a belső folyamatok láthatósága kritikus jelentőségű. Ezért a backendet felkészítettem arra, hogy úgynevezett callback függvényeket tudjon fogadni. Ezek a függvények egy olyan nézetet (view-t) kapnak a belső gépről, amelyen olyan metódusok vannak definiálva, amelyekkel a teljes AST-t be lehet járni, szemantikai információt fűzve az egyes csúcsokhoz. Így az egyes redukciós lépések könnyen naplózhatóak, az alkifejezések különböző szerepe pedig elkülöníthető.

Algoritmusokat dolgoztam ki a redukálható kifejezések (úgynevezett *redexek*) detektálására és strukturált adatként (JSON) történő továbbítására a frontend felé. Ezek az adatok teszik lehetővé, hogy a webes felület vizuálisan kiemelje a soron következő redukciós lépést.

2.6. A frontend fejlesztése

A frontend egy teljes mértékben kliensoldalon futó alkalmazás, amelyet Rust nyelvből WebAssembly-re (WASM) fordítottam.

Az Elm-architektúrára épülő reaktív felületen a felhasználó interaktívan gépelhet be kifejezéseket, kiválaszthatja a kívánt kalkulust és a kiértékelési stratégiát. A gördülékeny felhasználói élmény érdekében a háttérfolyamatokat Service Worker-ekbe (`worker.js`) szerveztem. Így a komplex és esetlegesen hosszú ideig futó redukciós fák kiértékelése nem fagyasztja le a felhasználói felületet, és a szoftver akár offline módban (PWA) is képes teljeskörűen funkcionálni.

2.7. Összefoglalás

A félév során sikeresen megvalósítottam a munkatervben kitűzött célokat. Áttanulmányoztam a releváns szakirodalmat, ami alapján megterveztem egy robusztus, modern architektúrát. Kialakítottam a nyelvtanokat feldolgozó parsereket, implementáltam mind a λ-, mind a SKI-kalkulus G-machine alapú redukciós motorját, és mindezt egy WebAssembly alapú, interaktív frontenddel kötöttem össze. A Podman konténeres orákulumra épülő Property-based testing gondoskodik a motorok megbízhatóságáról. Az elkészült *Combinator Explorer* eszköz így sikeresen tölti be funkcióját: vizuális formába önti a funkcionális programozás matematikai alapjait, hatékony és korszerű segédeszközt nyújtva a felsőoktatás számára.

3. Irodalom, és csatlakozó dokumentumok jegyzéke

3.1. A tanulmányozott irodalom jegyzéke

[1] Umberto Eco: *Hogyan írjunk szakdolgozatot?* 2000, Kairosz Kiadó. ISBN 9639137537.

3.2. A csatlakozó dokumentumok jegyzéke

Az alábbi kódrészlet a `lexor-reducer/src/core/engine.rs` fájlból származik. Azt a `reduce` függvényt és `match` blokkot mutatja be, amely az S, K, I, B és C kombinátorokhoz tartozó gráf-redukciós lépéseket (az új csomópontok allokálását és a mutatók átírását) végzi el az aréna-allokátoron belül.

```
        let result = match redex {
            NodeComb::I | NodeComb::K => Node::Indirection(args[0]),
            NodeComb::S => {
                let xz = self.arena.insert(Node::App(args[0], args[2]));
                let yz = self.arena.insert(Node::App(args[1], args[2]));
                Node::App(xz, yz)
            }
            NodeComb::B => {
                let yz = self.arena.insert(Node::App(args[1], args[2]));
                Node::App(args[0], yz)
            }
            NodeComb::C => {
                let xz = self.arena.insert(Node::App(args[0], args[2]));
                Node::App(xz, args[1])
            }
        };

        self.arena.replace(roots[0], result);
```

Listing 1. Kombinátorok redukciós szabályai a G-machine motorban

[1]